BIG DATA · Academic year 2026-2027

Assignment 2
Optimised Dense and Sparse Matrix Multiplication in C

Grado en Ciencia e Ingeniería de Datos
Universidad de Las Palmas de Gran Canaria

Carlos Falcón Brito

Repository: https://github.com/Cafabri/Assigment2

Abstract

This study measures how loop reordering, cache blocking, a transposed operand and compiler vectorisation change the time and memory of the triple-loop C multiplication of Assignment 1, and when a CSR sparse multiplication is faster or smaller than a dense one. All kernels are single-threaded and bit-identical, timed on an Intel Core i7-13700KF (median of 5–15 runs). Cache blocking was 16.4× faster than the baseline at $n = 4000$. AVX2 made it a further 2.5–2.8× faster with each build's best block size, and 2.19× with the same block size at $n = 2000$. At $n = 2000$ and with uniform nonzeros, CSR beat the scalar blocked kernel up to a density between 50 and 75% (about 55% by log interpolation) and the vectorised one up to about 33%. Its storage stayed smaller up to about 50%. The fill-in of the product, not the share of zeros alone, decides its advantage.

1 Introduction and objective

In Assignment 1 the `ijk` multiplication in C followed the n^3 model only up to $n = 1500$: between $n = 1500$ and 2000 its cost per multiply-add rose from 0.55 to 1.30 ns, attributed, without a direct measurement, to the column-by-column reading of B once it no longer fits in the L3 cache or the reach of the TLB. If so, changing only the order in which memory is read should remove most of the jump. Alternatively, a sparse format stores and multiplies only the nonzeros, but its indexing and the fill-in of the product can cancel that advantage [5]. The study asks two questions:

> *(A) How do loop reordering, cache blocking and a transposed operand change the runtime and memory of the* ***`ijk`*** *dense multiplication of Assignment 1, and how much of the change comes from compiler vectorisation?*
> *(B) Under which tested densities and nonzero distributions does a CSR sparse multiplication need less time or memory than a dense method on the same logical matrices?*

Factors varied: dense method, size n, block size b and build (Part A); density d of the inputs (percentage of nonzero entries; the sparsity is $100\% - d$), distribution of the nonzeros (uniform or band), n and method (Part B). **Outcomes:** kernel time, the separately timed conversion and transposition, the nonzeros of the product and the memory. **Kept fixed:** language, compiler and flags within each build, float64, input values, machine, one thread.

Hypotheses. Table 1 lists the predictions, derived from Section 2. P1, P2 and P4 were stated before any measurement. P3, P5 and P6 were adjusted after a pilot run, so they are expectations informed by early timings, not blind predictions.

Prediction	Statement (reason)
P1 Loop order	`ikj` is faster than `ijk` at every $n \geq 500$ and its cost per multiply-add has no jump between $n = 1500$ and 2000 (B and C read with unit stride)
P2 Blocking	Blocked is faster than `ikj` once B no longer fits in the L3, $n \geq 2000$ (tiles of B reused from the cache)
P3 Transposed	Faster than `ijk` but slower than `ikj` (unit stride, but one serial chain of n additions per element of C)
P4 Vectorisation	AVX2 speeds up only the loops gcc reports as vectorised (`ikj`, blocked), by at most 4× (4 doubles per instruction)
P5 Sparse time	CSR is faster than scalar blocked up to at least 50% density, more so for band than for uniform inputs (about n^3d^2 multiply-adds, plus costs that grow with the nonzeros of C, which fill in faster for uniform inputs)
P6 Sparse memory	CSR needs less memory than the dense arrays only while the product stays sparse; for uniform inputs with $d \geq 5\%$ it needs more (12 bytes per nonzero against 8 per dense entry)

Table 1: Predictions, checked with the cost per multiply-add and the ratios of medians (P1–P5) and with the computed storage and peak working set (P6).

2 Background

Dense optimisations. The triple loop performs n^3 multiply-adds whatever the loop order [1]; the order only changes which memory the innermost loop reads. In `ijk` the index k walks down a column of B, $8n$ bytes between accesses. Interchanging the inner loops (`ikj`) walks along rows of B and C, so every 64-byte line serves 8 multiply-adds [2], but all of B is still read once per row of C. *Cache blocking* splits the loops into $b \times b$ tiles, so a tile of B is reused while it is cached; the best b depends on the caches [4]. Transposing B once, in $O(n^2)$, lets the `ijk` dot product read both operands with unit stride.

Rounding and vectorisation. Floating-point addition is not associative [3], so all kernels add the products of each $C[i][j]$ in increasing k and give results bit-identical to `ijk`. *Vectorisation* executes one instruction on several values: 4 doubles with AVX2, 2 with SSE2. gcc vectorises loops at `-O2` with a `very-cheap` cost model; `-fvect-cost-model=dynamic` also allows loops that need a run-time overlap check [7], and `-mavx2` allows AVX2 instructions [8]. The inner loop of `ikj` can be vectorised over j without changing any sum; a dot product cannot, unless the additions may be reordered.

Sparse matrices and CSR. The Compressed Sparse Row format stores the nonzeros row by row (values and column indices) plus $n + 1$ row offsets [6]: with 8-byte values, 4-byte indices and 8-byte offsets, $12z + 8(n + 1)$ bytes for z nonzeros, against $8n^2$ dense. Gustavson's algorithm computes row i of $C = AB$ as the sum of the rows k of B scaled by the nonzeros $A[i][k]$ [5]: about n^3d^2 multiply-adds for density d, each with an indirect access. The product fills in: for uniform nonzeros $C[i][j]$ is nonzero with probability $1 - (1 - d^2)^n$ (99% for $d = 5\%$ at $n = 2000$), while a band product ($|i - j| \leq 2w$ for inputs with $|i - j| \leq w$) has at most about twice the input density.

3 Methodology

3.1 Implementations and reused material

The baseline is the validated C implementation of Assignment 1 (`https://github.com/Cafabri/Assigment1`, commit `0785887`): its kernel, renamed, and its random generator, seeds, timer, batching rule, processor affinity and peak-memory measurement are reused unchanged. Only C was continued (the fastest language of Assignment 1), so that no language effect mixes with the optimisation and sparse-storage effects. All kernels are single-threaded own code and call no numerical library:

- `ijk` (baseline); `ikj` (the j and k loops interchanged); **blocked**, `ikj` on $b \times b$ tiles with tile loops ii, kk, jj (kk outside jj, so each element still receives its products in increasing k; partial edge tiles handled with bounds); `transposed`, the `ijk` loop on B^T, prepared before the call (the transposition, `transpose`, is timed separately).
- `csr`: Gustavson's algorithm. A symbolic pass counts the nonzeros of each row of C, so C is allocated once; a numeric pass accumulates each row in a dense array with n markers, and sorts its column indices by a scan of the markers when the row has more than $n/16$ nonzeros, otherwise with `qsort`. The nonzeros of C are bit-identical to `ijk`, whose zero factors add only exact $+0$. `convert`: dense $\rightarrow$ CSR conversion of A and B, timed separately.

3.2 Input data

Values are those of Assignment 1: SplitMix64 with seed 42 for A and 43 for B, uniform in $[0, 10)$. The generator is counter-based, so position $p = i \cdot n + j$ has the same value at every density. Nonzero positions: **dense**, all (the matrices of Assignment 1); **uniform**, each position independently when a second SplitMix64 stream (seeds 1042 and 1043) gives a number below d; **band**, positions with $|i - j| \leq w$, w the smallest half-width whose band holds dn^2 positions, each kept with probability dn^2/(band positions) (above 0.95 from $d = 1\%$; `results/patterns.csv`). Sparser matrices are subsets of denser ones. The sparse method generates its inputs directly in CSR, so large sparse inputs never exist as dense arrays; the program checks that this equals converting the dense matrix.

3.3 Environment

CPU: Intel Core i7-13700KF, 8 P-cores with 2 threads each (logical processors 0–15) and 8 E-cores; 48 KB L1 data and 2 MB L2 per P-core, 30 MB shared L3; 31.8 GiB RAM; Windows 11 Pro, power plan *Balanced*. The runner and every benchmark process are restricted to the P-cores. **Compiler:** gcc 16.1.0 (MSYS2/MinGW64) with three builds: `scalar`, `-O2 -ffp-contract=off` (the flags of Assignment 1); `sse2`, which adds `-fvect-cost-model=dynamic`; and `avx2`, which also adds `-mavx2`. **Analysis:** CPython 3.11.9, NumPy 2.2 and SciPy 1.15 (reference results), pandas 2.2, matplotlib 3.10 (`results/*/environment.txt`). `-O3`, which could interchange the loops by itself, and fused multiply-adds are avoided in every build [7]. The gcc reports (`-fopt-info-vec`, `results/*/vectorisation_report.txt`) show no vectorised kernel loop in the scalar build; the inner loops of `ikj` and blocked (and the counting loop of the conversion) vectorised with 16-byte (SSE2) or 32-byte (AVX2) vectors behind a run-time overlap check; and the dot products of `ijk` and `transposed` vectorised in no build (that needs `-fassociative-math` [7]), so `transposed` is a control.

3.4 Measurement protocol

`run_experiment.py` compiles the builds and runs each configuration in a separate process, never two at once. **Block size:** blocked, $b \in \{16, 32, 64, 128, 256\}$, $n \in \{1000, 2000\}$; the b^* of each build is the one with the lowest median at $n = 2000$ (rule fixed before running). **Dense (Part A):** `ijk`, `ikj`, blocked, `transposed` and `transpose` (scalar), `ikj`, blocked and `transposed` (AVX2), $n \in \{100, 200, 500, 1000, 1500, 2000, 3000, 4000\}$. This sequence was fixed before measuring and ends at $n = 4000$ by design, to keep the whole study at about 4 hours of benchmark processes. **Density (Part B):** $n = 2000$, $d \in \{0.1, 0.5, 1, 2, 5, 10, 20, 30, 50, 75, 100\}\%$, both distributions: `csr` and `convert` (scalar; the CSR kernel has no vectorised loop), blocked in both builds, and `ijk` at $d \in \{0.1, 10, 100\}\%$. **Size (Part B):** `csr` for $n = 1000$ to 32 000 (doubling) at $d \in \{0.1, 1\}\%$, both distributions, and scalar blocked up to $n = 4000$.

A **second measurement session (27 October 2026)**, with the same programs, protocol and budget (`run_experiment.py --extension`, 46.2 min, `results/extension/`), added: the `sse2` build, which separates enabling vectorisation from the vector width (block sweep, and `ikj` and blocked with its b^* at every dense size); series up to the budget (`csr` at $n = 64\,000$ for the four size inputs, and scalar `ijk` at $n = 8000$, whose run time a cubic extrapolation placed beyond 15 minutes, an estimate, not a measurement); and three drift references at $n = 2000$ (scalar and AVX2 blocked, scalar `ikj`) to compare the two sessions.

The **budget** is 15 minutes per run, as in Assignment 1 (the process is killed: TIMEOUT), and 16 GiB of matrix storage per process, checked before allocating (for CSR, after the symbolic pass: MEMORY_LIMIT); larger sizes of a failed series are SKIPPED. **Repetitions:** each process makes 2 warm-up and 5 measured runs; configurations whose first process had a median below 60 s are repeated in 3 interleaved processes (15 runs). Each configuration is summarised by the median and the IQR. A first call shorter than 10 ms is a separate check call, and the runs are then timed in batches of at least 10 ms (time per call reported).

The **timer** (`QueryPerformanceCounter`) covers only the kernel call, with different boundaries for the two families: dense kernels receive their inputs and a zeroed C prepared outside it, as in Assignment 1, while the CSR timer includes the symbolic pass, the allocation and sorting of C and, in batched runs, freeing the previous outputs. This asymmetry favours the dense kernels. The CSR inputs are already in CSR; the conversion is reported separately and added only where stated. Every check call and unbatched run writes the checksum of its result, so no call can be removed by the compiler.

Memory. `GetProcessMemoryInfo` gives the *peak working set* of each process, which holds only the arrays of its own method. The *storage* is computed: $24n^2$ bytes for the dense kernels ($32n^2$ with B^T); for `csr`, $12z + 8(n + 1)$ bytes per matrix (A, B, C) plus $12n$ bytes of work space.

3.5 Correctness

(1) At start-up every kernel checks identity, zero, 2×2 and 7×7 sparse cases with an empty row and partial tiles. (2) For $n = 50$ and 257 (prime, so every b leaves partial tiles) and five inputs (dense; uniform and band at 5% and 30%), every kernel, block size and build is compared with NumPy, and the CSR nonzeros with SciPy, on independently regenerated inputs (relative tolerance 10^{-11}, above the rounding bound nu [3] of 4.4×10^{-13} at $n = 4000$; reference zeros exact). (3) These results must equal scalar `ijk` bit for bit. (4) In every configuration the checksum must be identical across runs and agree with NumPy/SciPy, and the nonzeros of A, B and C must equal SciPy's (above 10^8 nonzeros only agreement between runs).

4 Results

4.1 Correctness and variability

All checks passed (`results/correctness.csv`, `checksums.csv`). The largest error against NumPy was 1.9×10^{-15}, and all 25 kernels were bit-identical to scalar `ijk`. The checksums were identical across runs in all 205 configurations. The baseline reproduces Assignment 1: `ijk` took 10.9 s at $n = 2000$, against 10.4 s there.

The IQR is at most 3% of the median in 78.5% of the configurations of the first session and in all of the second. The largest IQR is that of the transposition alone (up to 33.4%, calls of a few ms). The process medians of `ijk` at $n = 2000$ (10.49–11.09 s) make the speedups at that size uncertain by about 10%.

4.2 Part A: dense methods

Block size (Table 2). The rule gives $b^* = 32$ (scalar), 64 (SSE2) and 256 (AVX2). At $n = 2000$ the scalar differences are small and not monotonic, with the worst b 1.19× slower. In the AVX2 build the time falls as b grows, and $b = 16$ is 1.85× slower than $b = 256$.

	Scalar build		SSE2 build		AVX2 build	
b	**n = 1000**	**n = 2000**	**n = 1000**	**n = 2000**	**n = 1000**	**n = 2000**
16	0.252 (0.9%)	2.02 (0.5%)	0.180 (0.3%)	1.44 (0.2%)	0.147 (1.3%)	1.19 (1.3%)
32	0.222 (0.3%)	1.77 (0.7%)	0.143 (0.6%)	1.16 (0.3%)	0.101 (2.1%)	0.810 (0.7%)
64	0.261 (3.1%)	2.11 (1.4%)	0.125 (0.6%)	1.00 (0.3%)	0.0876 (2.6%)	0.707 (1.0%)
128	0.247 (1.2%)	1.98 (0.8%)	0.135 (1.7%)	1.08 (0.7%)	0.0815 (2.3%)	0.661 (1.6%)
256	0.236 (0.7%)	1.92 (1.1%)	0.144 (1.1%)	1.15 (0.4%)	0.0815 (2.3%)	0.646 (0.8%)

Table 2: Median time of the blocked kernel (s), IQR as a percentage of the median in parentheses; 15 runs in 3 processes.

Loop order and blocking (Table 3, Figure 1). `ikj` is faster than `ijk` at every size, by 8.6× at $n = 4000$. From $n = 1500$ to 2000, the cost per multiply-add of `ijk` jumps from 0.52 to 1.36 ns; that of `ikj` rises less, from 0.25 to 0.36 ns. The blocked kernel stays at 0.22 ns per multiply-add up to $n = 4000$ (fitted exponent 3.01). For $n \geq 2000$ it is 1.61–1.90× faster than `ikj`.

Transposed operand. `transposed` is within 4% of `ijk` at $n = 200$ and 500. It is faster from $n = 1000$, by 7.2× at 4000, but always slower than `ikj` (1.2–1.6× from $n = 200$). The transposition itself takes at most 0.5% of the kernel time.

n	ijk (s)	ikj (s)	blocked (s)	transposed (s)	transpose (s)
100	3.01×10^{-4} (5.2%)	2.48×10^{-4} (1.9%)	2.26×10^{-4} (2.3%)	2.57×10^{-4} (4.5%)	3.85×10^{-6} (7.9%)
200	2.72×10^{-3} (1.2%)	1.80×10^{-3} (3.0%)	1.76×10^{-3} (2.3%)	2.84×10^{-3} (3.0%)	1.07×10^{-5} (3.5%)
500	0.0468 (1.6%)	0.0285 (1.0%)	0.0274 (2.1%)	0.0459 (2.0%)	2.13×10^{-4} (9.0%)
1000	0.467 (1.3%)	0.232 (1.0%)	0.219 (2.1%)	0.378 (1.6%)	9.00×10^{-4} (3.3%)
1500	1.77 (9.4%)	0.858 (1.4%)	0.741 (1.0%)	1.35 (2.2%)	2.40×10^{-3} (20.6%)
2000	10.9 (4.7%)	2.84 (2.7%)	1.77 (1.1%)	3.76 (3.4%)	6.40×10^{-3} (33.4%)
3000	73.8 (0.2%)	10.7 (1.4%)	5.97 (0.8%)	13.3 (0.5%)	0.0208 (1.3%)
4000	234 (1.2%)	27.0 (2.8%)	14.2 (1.6%)	32.4 (1.7%)	0.0477 (9.8%)
8000	TIMEOUT	—	—	—	—

Table 3: Scalar build: median time per multiplication (s), IQR as a percentage of the median in parentheses. *transpose* is the transposition of B alone, not included in *transposed*. 15 runs in 3 processes, except `ijk` at $n \geq 3000$ (5 runs in one process). For $n \leq 200$ each run is a batch of calls lasting at least 10 ms.

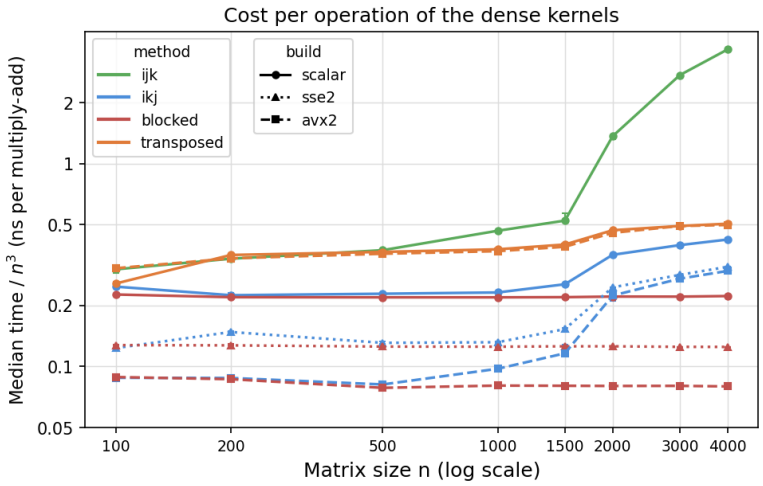


Figure 1: Median time divided by n^3 (ns per multiply-add); colour: method, line style: build. Bars: IQR (Q1–Q3), mostly hidden by the markers. A runtime proportional to n^3 gives a horizontal line.

Vectorisation (Table 4). With the b^* of each build, AVX2 makes the blocked kernel 2.54–2.79× faster at every size. With the same $b = 32$ at $n = 2000$ the gain is 2.19× (Table 2), so part of it comes from the larger tiles that the vector build prefers. The SSE2 build splits the gain into two steps. For the blocked kernel, enabling vectorisation gives 1.75–1.78× and widening the vectors a further 1.55–1.60×. For `ikj` the widening step falls from 1.60× at $n = 500$ to 1.04–1.05× from $n = 3000$. `transposed`, which is not vectorised, gains nothing. The AVX2 blocked kernel is 45.8× faster than `ijk` at $n = 4000$.

	`ikj`			blocked (b^* of each build)		
n	**Enable**	**Widen**	**Total**	**Enable**	**Widen**	**Total**
100	2.01	1.4	2.81	1.78	1.44	2.55
200	1.52	1.69	2.56	1.73	1.47	2.54
500	1.75	1.6	2.8	1.75	1.6	2.79
1000	1.76	1.35	2.38	1.75	1.55	2.72
1500	1.66	1.32	2.19	1.75	1.56	2.73
2000	1.45	1.1	1.59	1.76	1.57	2.76
3000	1.4	1.04	1.46	1.77	1.56	2.75
4000	1.36	1.05	1.43	1.78	1.57	2.79

Table 4: Gain of each build step (ratio of medians). Enable: scalar → SSE2 (2 doubles per instruction); Widen: SSE2 → AVX2 (2 → 4 doubles); Total: scalar → AVX2. The blocked kernel uses $b = 32$, 64 and 256 in the three builds; at a fixed $b = 32$ the total at $n = 2000$ is 2.19 (Table 2).

Drift between sessions. Three references were repeated at $n = 2000$, with 15 runs in each session. The AVX2 blocked median moved by +0.1%, inside the first IQR. The scalar blocked median moved by −0.4% (1.769 to 1.762 s), and scalar `ikj` by −4.2% (2.842 to 2.724 s). Ratios between the SSE2 build and the other two are therefore uncertain by under 1% for the blocked kernel and up to about 4% for `ikj`.

Memory. The peak working set of `ijk`, `ikj` and blocked is identical in every build: 24.0 bytes per element, the three float64 matrices. `transposed` needs 32.0 bytes per element, because it also holds B^T.

4.3 Part B: density at $n = 2000$

Dense methods and fill-in. The dense kernels do not depend on the zeros: across the 22 inputs, the blocked median varies by 1.6% in each build (Table 5). For uniform inputs the density of C follows $1 - (1 - d^2)^n$ and reaches 99.3% at $d = 5\%$. For band inputs it stays about twice the input density and reaches 100% only at $d = 75\%$ (Figure 2, left).

Time (Table 5, Figure 3). CSR is faster than the scalar blocked kernel up to $d = 50\%$ in both distributions. Its advantage is larger for band inputs at every density below the crossover, for example 1000× against 187× at $d = 1\%$. Against the scalar blocked kernel, the crossover lies between 50 and 75% density (about 55% uniform and 57% band by log interpolation). Against the AVX2 kernel it lies between 30 and 50% (about 33% and 35%). Divided by n^3d^2, the CSR time is 0.65–0.84 ns for $d \geq 20\%$ but grows at low density, up to 26.6 ns (uniform, $d = 0.5\%$).

Conversion. Converting A and B takes 7.8–53 ms. At $d = 0.1\%$ this is 45× (uniform) and 85× (band) the CSR kernel time. From $d = 10\%$ it is under 20% of it. Including it moves the crossovers by at most 1.2 percentage points.

Memory (Figure 2, right). The CSR storage of A, B and C stays below the $24n^2$ bytes of the three dense matrices up to about 50% density (uniform) and 56% (band). At 100% it is 1.5× larger. At $d = 5\%$, with an almost full product, the uniform CSR matrices still need 0.55× the dense storage. The CSR peak working set grows from 4.3 to 141.5 MiB, against 95.6 MiB for the dense kernel.

	Density (%)					Dense blocked (s)		Scalar blocked /	
d (%)	**A**	**B**	**C**	**CSR (s)**	**Conv. (s)**	**scalar**	**AVX2**	**CSR**	**CSR + conv.**
Uniform nonzeros									
0.1	0.1	0.099	0.2	1.73×10^{-4}	7.84×10^{-3}	1.76	0.640	10100	219
0.5	0.49	0.5	4.81	5.31×10^{-3}	8.76×10^{-3}	1.77	0.646	333	126
1	0.99	1	18	9.52×10^{-3}	8.98×10^{-3}	1.78	0.645	187	96
2	1.99	2	55	0.0295	1.00×10^{-2}	1.77	0.642	59.9	44.7
5	4.99	5	99.3	0.0641	0.0139	1.76	0.643	27.5	22.6
10	9.98	10	100	0.106	0.0207	1.79	0.647	16.9	14.1
20	20	20	100	0.268	0.0333	1.76	0.643	6.58	5.85
30	30	30	100	0.536	0.0420	1.76	0.641	3.29	3.05
50	50	50	100	1.43	0.0529	1.76	0.639	1.23	1.18
75	75	75	100	3.38	0.0379	1.76	0.648	0.522	0.516
100	100	100	100	6.04	0.0175	1.76	0.642	0.292	0.291
Band nonzeros									
0.1	0.1	0.099	0.15	9.32×10^{-5}	7.90×10^{-3}	1.76	0.642	18900	220
0.5	0.5	0.5	1.03	7.66×10^{-4}	7.97×10^{-3}	1.77	0.642	2310	202
1	1	1	2.03	1.76×10^{-3}	8.03×10^{-3}	1.76	0.638	1000	180
2	2	2	4.01	5.11×10^{-3}	8.37×10^{-3}	1.76	0.641	346	131
5	5	5	9.98	0.0172	8.57×10^{-3}	1.77	0.641	102	68.4
10	10	10	19.6	0.0572	9.26×10^{-3}	1.76	0.640	30.7	26.4
20	20	20	37.8	0.211	9.89×10^{-3}	1.76	0.645	8.36	7.98
30	30	30	54.7	0.465	0.0109	1.77	0.644	3.8	3.72
50	50	50	82.9	1.30	0.0128	1.77	0.645	1.37	1.35
75	75	75	100	3.27	0.0153	1.76	0.643	0.54	0.537
100	100	100	100	6.06	0.0175	1.77	0.640	0.291	0.291
Largest IQR (%)				3.7	7.9	3.7	2.8		runs: 15

Table 5: $n = 2000$. Requested density d; measured density of A, B and C; median times of the CSR kernel, of the conversion of A and B and of the dense blocked kernel; ratio of the scalar blocked time to the CSR time without and with the conversion (above 1: CSR is faster). Last row: largest IQR (% of the median) of each time column.

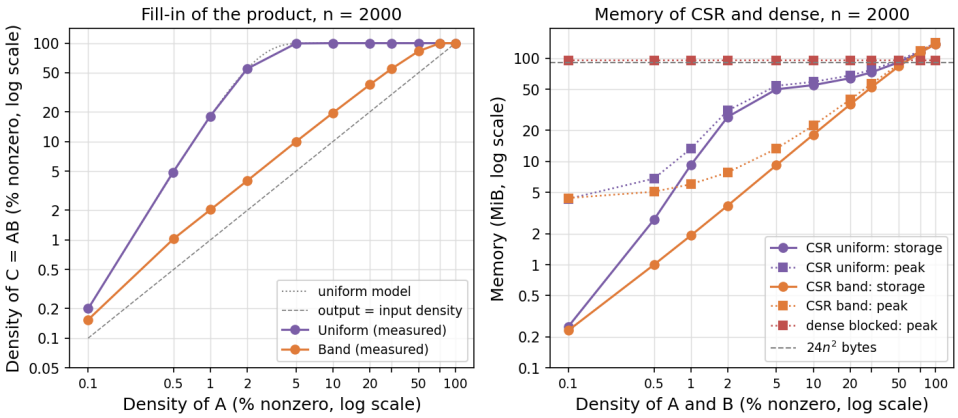


Figure 2: $n = 2000$, log-log. Left: density of $C = AB$ vs density of the inputs; dotted line: uniform model $1 - (1 - d^2)^n$. Right: computed CSR storage (A, B, C and work space) and peak working set of the CSR and dense blocked processes; dashed line: $24n^2$ bytes.

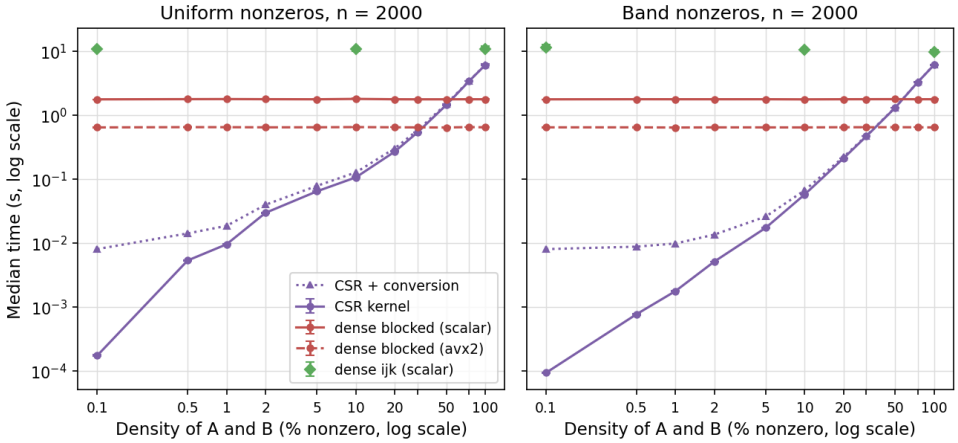


Figure 3: Median time vs density of A and B at $n = 2000$ (log-log). Left: uniform nonzeros; right: band nonzeros. Bars: IQR (Q1–Q3), mostly hidden by the markers. `ijk` was run at three densities only.

4.4 Part B: matrix size and largest feasible size

Table 6 gives the CSR time from $n = 1000$ to 32 000. For the uniform inputs the fitted exponent is 3.33 (0.1%) and 2.74 (1%). For the band inputs it is 2.46 and 2.52. The uniform products fill in as n grows: at $d = 1\%$, from 9.4% to 95.9% nonzeros. The band products keep a constant density, and the CSR storage follows the nonzeros of C. Where both ran ($n \leq 4000$), CSR is 158–24 300× faster than the scalar blocked kernel.

Two series reached the budget (Table 7). `ijk` at $n = 8000$ was killed after 900 s of its first run. CSR uniform 1% at $n = 64\,000$ stopped after the symbolic pass, because its storage would need 46.6 GiB. The other three CSR inputs finished at $n = 64\,000$. For `ikj`, blocked and `transposed`, $n = 4000$ is the end of the size sequence fixed by design, not a failure: no larger size was attempted.

	Uniform 0.1%		Uniform 1%		Band 0.1%		Band 1%		Blocked
n	**CSR (s)**	**MiB**	**CSR (s)**	**MiB**	**CSR (s)**	**MiB**	**CSR (s)**	**MiB**	**(s)**
1000	1.32×10^{-5}	0.1	1.39×10^{-3}	1.3	9.72×10^{-6}	0.1	3.83×10^{-4}	0.5	0.220
2000	1.73×10^{-4}	0.3	9.40×10^{-3}	9.2	9.38×10^{-5}	0.2	1.77×10^{-3}	1.9	1.77
4000	1.61×10^{-3}	1.2	0.0662	64.0	5.79×10^{-4}	0.9	0.0102	7.5	14.1
8000	0.0159	7.6	0.488	418	2.58×10^{-3}	3.2	0.0561	29.6	—
16000	0.154	53.0	3.08	2 397	0.0110	12.4	0.345	118	—
32000	1.45	394	17.3	11 477	0.0617	48.3	2.35	469	—
64000	8.63	3 004	MEMORY_LIMIT	(47 737)	0.326	190	17.5	1 876	—

Table 6: CSR at fixed density: median time and computed storage of A, B, C and work space (MiB); the peak working set was 4.1–4.3 MiB above the storage. In parentheses, the storage the program computed before stopping. Last column: scalar blocked kernel on the uniform 1% inputs (its time does not depend on the input). 15 runs in 3 processes; largest IQR 8.3% of the median.

Series	**Largest OK n**	**Longest median (s)**	**Peak (MiB)**	**First n over budget**
`ijk` (scalar)	4000	234	370	8000 (TIMEOUT)
`ikj` (scalar)	4000	27.0	370	none
`ikj` (sse2)	4000	19.8	370	none
`ikj` (avx2)	4000	18.9	370	none
`blocked` (scalar)	4000	14.2	370	none
`blocked` (sse2)	4000	7.99	370	none
`blocked` (avx2)	4000	5.10	370	none
`transposed` (scalar)	4000	32.4	492	none
`transposed` (avx2)	4000	31.9	492	none
CSR, uniform 0.1%	64000	8.63	3 008	none
CSR, uniform 1%	32000	17.3	11 481	64000 (MEMORY_LIMIT)
CSR, band 0.1%	64000	0.326	194	none
CSR, band 1%	64000	17.5	1 880	none

Table 7: Largest size with status OK under the budget of 15 minutes per run and 16 GiB of matrix storage, its longest median and largest peak working set, and the first size that exceeded the budget ("none": no size of the series exceeded it). For the dense methods other than `ijk`, $n = 4000$ is a design limit (the last declared size), not a failure.

5 Discussion

5.1 Evaluation of the hypotheses

Table 8 compares each prediction with the results; Sections 5.2 and 5.3 interpret them.

Prediction	Observed	Verdict
P1 Loop order	`ikj` faster at every $n \geq 500$, but its cost per multiply-add still rises from 0.25 to 0.36 ns between $n = 1500$ and 2000	Partly confirmed (smaller jump, not none)
P2 Blocking	1.61–1.90× faster than `ikj` for $n \geq 2000$; constant 0.22 ns per multiply-add	Confirmed
P3 Transposed	Slower than `ikj` at every size; faster than `ijk` only from $n = 1000$	Partly confirmed
P4 Vectorisation	Blocked 2.5–2.8× (b^* of each build), 2.19× at fixed b; the non-vectorised `transposed` within 4%	Confirmed
P5 Sparse time	Crossovers between 50 and 75% (about 55% uniform, 57% band); band advantage larger at every density below them	Confirmed
P6 Sparse memory	CSR storage smaller than dense up to about 50% (uniform) and 56% (band), including uniform inputs with $5\% \leq d \leq 50\%$	Refuted (threshold wrong)

Table 8: Verdict for each prediction of Table 1.

5.2 Effect of the dense optimisations

The dense kernels perform the same operations in the same order, so their differences come only from memory access order and instructions. The explanations below are hypotheses consistent with the timings; no hardware counters or profiler were used.

Memory access. At $n = 2000$, B (32 MB) no longer fits in the 30 MB L3. Between $n = 1500$ and 2000, the cost per operation of `ijk` grows 2.6× and that of `ikj` only by 40%. Since `ikj` uses every cache line of B fully, its smaller jump is consistent with the L3-capacity explanation of Assignment 1: B is read again from memory for every row of C, at about 22.5 GB/s. The larger jump of `ijk` suggests that its stride adds to it; the shares of cache and TLB remain open.

Cache blocking removes the dependence on n, as expected if tiles of B are reused from the cache [4]. Three tiles take 24 KB with $b = 32$ (48 KB L1) and 1.5 MB with $b = 256$ (2 MB L2). Small tiles may be too short for the AVX2 vector loop.

Transposed operand. Up to $n = 500$, where B still fits in the cache, `ijk` and `transposed` both cost 0.34–0.37 ns per multiply-add, and `ikj`, whose consecutive additions go to different elements, 0.23 ns. This is consistent with dot products limited by the latency of one serial chain of additions [2].

Vectorisation. The loop order is what allows it: `ikj` updates independent elements and is vectorised, the dot product is not. Over `ijk` at $n = 4000$, blocking alone gives 16.4× and vectorisation with the loop reordering it needs 12.4×. Combined, they give 45.8×. Part of the blocked gain comes from the larger b^* of the vector builds (2.19× at fixed b against 2.76×). For the blocked kernel, 54–56% of the gain (in log scale) comes from enabling vectorisation and the rest from the wider vectors. For `ikj` at $n \geq 3000$, SSE2 already obtains 87–89% of it. This is consistent with `ikj` being limited by memory there, where wider arithmetic would not shorten the wait. P4 holds, with that refinement.

5.3 Effect of sparse storage

Crossover. Unlike the dense optimisations, which keep the n^3 operations, CSR removes operations but pays more for each. For $d \geq 20\%$ its cost per product is 2.9–3.8× that of the scalar blocked kernel and 8.1–10× that of the vectorised one (a column index, a marker check and a scattered update per product). A constant-cost model, with the uniform CSR cost at $d = 50\%$ (0.717 ns), puts the crossover where d^2 equals the ratio of costs. It is coherent with both measured crossovers, scalar and AVX2, to within one percentage point. As it is calibrated at $d = 50\%$, next to the crossover, this is a consistency check, not an independent validation; the crossover depends on the dense alternative.

Low density and distribution. At low density the cost per product grows, because the products fall as d^2 while the costs per row and per nonzero of C do not. The uniform peak at $d = 0.5\%$ is consistent with the sorting rule. There the rows of C have about 96 nonzeros, fewer than $n/16 = 125$, and are sorted with `qsort`; at $d = 1\%$ a scan of the markers is used. The distribution acts through the fill-in. At $d = 1\%$ both perform about the same number of products, but the uniform product has 18% nonzeros and the band product 2%, and the band kernel is 5.4× faster.

Conversion and memory. At $d = 0.1\%$ the conversion is bounded below by reading the two dense matrices. Its uniform peak at $d = 50\%$ is consistent with an unpredictable branch on `v != 0`. P6 compared only the product: a full CSR C needs 12 bytes per entry against 8 for a dense C. Over all three matrices, the CSR total stays below the dense total up to about 50% density (uniform), because A and B are sparse.

Scaling with n and limit. For band inputs, fixed per-row costs are amortised: the time per product of band 1% falls from 3.83 to 0.716 ns up to $n = 32\,000$. For uniform 0.1% it grows from 13.2 to 44.2 ns, consistent with `qsort` on rows that stay below $n/16$ nonzeros. Its fall to 32.9 ns at $n = 64\,000$ is not explained by this. The limit of CSR is set by the nonzeros of the product. At $n = 32\,000$ the uniform 1% product has 96% nonzeros, so almost all of the 11.2 GiB of CSR storage is C, against 22.9 GiB for three dense matrices. At $n = 64\,000$ the CSR total would need 46.6 GiB, and three dense matrices 91.6 GiB. No dense run was attempted at $n = 32\,000$: cubic extrapolation of the blocked kernel gives about 2.0 hours per run (estimated, not measured).

5.4 Observed bottlenecks

Inferred from the timings: `ijk`, the strided reads of B beyond the L3; `ikj`, reading B from memory for every row of C ($n \geq 2000$); `transposed`, the latency of one serial chain of additions; blocked, the arithmetic, reduced only by vectorisation (0.22 to 0.080 ns); CSR, the indirect access of every product, the sorting of C at low density and the fill-in of C, which sets its memory; the conversion, reading the dense arrays (dominant only below about 1% density).

5.5 Limitations

One computer, compiler and set of flags, without fixed frequency; the cache, TLB, latency, sorting and branch explanations are inferred from timings only. Five square block sizes, without register blocking. The crossovers are interpolated between 11 densities, at one size and for two distributions, and depend on n through the fill-in. The CSR and dense timers have different boundaries (Section 3.4). `ijk` at $n \geq 3000$ ran in one process. The drift references moved by up to 4.2% between sessions, which bounds comparisons across them. The largest feasible size was bracketed only for `ijk` (4000 to 8000) and CSR uniform 1% (32 000 to 64 000).

6 Conclusions

For these programs, inputs and machine:

- Reordering memory accesses makes the dense multiplication much faster with the same operations and memory, and blocking removes the jump of Assignment 1.
- Vectorisation multiplies the effect of the optimisations it applies to. Wider vectors help the blocked kernel but hardly `ikj` at large n, consistent with `ikj` being limited by memory there.
- At $n = 2000$, CSR beats the scalar blocked kernel up to a density between 50 and 75% (about 55% by log interpolation), and the vectorised one up to about a third. Its size limit is set by the fill-in of the product: under 16 GiB, the uniform 1% inputs ran only up to $n = 32\,000$.

Sparse storage saves operations at a higher cost per operation: its advantage depends on the density, the nonzero distribution and the dense alternative, not on the share of zeros alone.

Open questions. The cache/TLB split of the dense slowdowns (no hardware counters were used); the drop of the uniform 0.1% cost at $n = 64\,000$; a comparison with an optimised sparse library; and the untuned sorting threshold of $n/16$ nonzeros per row.

Reproducibility

The repository holds the code, the commands (`README.md`) and the raw data of both sessions. The study is reproduced with `run_experiment.py` (3.3 h), its `--extension` (46.2 min) and the scripts in `analysis/`.

Use of generative AI

Claude Opus 5.5 (Anthropic), used through Claude Code, served as a support tool during this assignment: to help write and debug the benchmark program, the experiment runner and the analysis scripts, including those of the second measurement session, and to improve the structure and wording of the report. All the measurements come from running the code on the machine described in Section 3.3. The author reviewed the code and the AI-generated text, checked the numbers against the raw data and the references, and is responsible for the conclusions.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[2] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2019.

[3] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Comput. Surv.*, vol. 23, no. 1, pp. 5–48, Mar. 1991, doi: 10.1145/103162.103163.

[4] M. D. Lam, E. E. Rothberg, and M. E. Wolf, "The cache performance and optimizations of blocked algorithms," in *Proc. 4th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS IV)*, 1991, pp. 63–74, doi: 10.1145/106972.106981.

[5] F. G. Gustavson, "Two fast algorithms for sparse matrices: multiplication and permuted transposition," *ACM Trans. Math. Softw.*, vol. 4, no. 3, pp. 250–269, 1978, doi: 10.1145/355791.355796.

[6] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA, USA: SIAM, 2003.

[7] Free Software Foundation, "Options that control optimization," in *Using the GNU Compiler Collection (GCC)*. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html. Accessed: Oct. 26, 2026.

[8] Free Software Foundation, "x86 options," in *Using the GNU Compiler Collection (GCC)*. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/x86-Options.html. Accessed: Oct. 26, 2026.